```
>> class Game
>>   def self.play(character)
>>     character.jump
>>   end
>> end
=> :play
```

When you're testing this class, first you'll *arrange* your test double:

```
>> mario = double('Mario')
=> #<Double "Mario">
```

...*assert* that it will receive the :jump message:

```
>> expect(mario).to receive(:jump)
=> #<RSpec::Mocks::MessageExpectation #<Double "Mario">.jump(any arguments)>
```

...and finally *act* by playing the game:

```
>> Game.play(mario)
=> nil
```

It feels a bit backwards to have to assert before acting. Spies are one way to restore the traditional flow. All you have to do is change the receive expectation to have_received, and then you can move your expectation to the end:

```
>> mario = double('Mario').as_null_object
=> #<Double "Mario">
>> Game.play(mario)
=> #<Double "Mario">
>> expect(mario).to have_received(:jump)
=> nil
```

Notice that we've had to define mario as a null object. If it had been a regular, strict double, you would have gotten a failure when you called the play method (because play would have sent it an unexpected jump message).

When you spy on objects with have_received, you'll either need to use null objects or explicitly allow the expected messages:

```
>> mario = double('Mario')
=> #<Double "Mario">
>> allow(mario).to receive(:jump)
=> #<RSpec::Mocks::MessageExpectation #<Double "Mario">.jump(any arguments)>
>> Game.play(mario)
=> nil
>> expect(mario).to have_received(:jump)
=> nil
```

Having to spell out the same message twice (once before calling Game.play and once afterward) somewhat defeats the purpose of spying. It's easier